

Fundamentals of Data Science

Lecture 1 - DS workflow, tech setup & code structure

Adrian Ochs ¹ and Christian Roerig ²

Michaelmas 2026

¹Co-founder @ Quantum Leap, ex-QuantCo

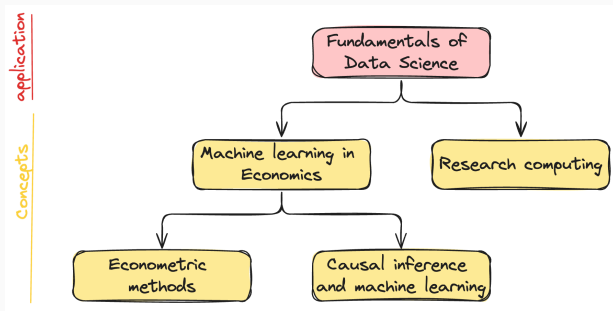
²Data Scientist/ML Engineer @ QuantCo

Hello in pictures



Course Overview

This course is very applied and builds on the concepts of your other modules



The interaction of your modules

Course objectives: What you will achieve

This does not mean that we remain superficial. By the end of this programme, you will (hopefully) be equipped to:

Course objectives: What you will achieve

This does not mean that we remain superficial. By the end of this programme, you will (hopefully) be equipped to:

- Develop and maintain a professional-grade code base

Course objectives: What you will achieve

This does not mean that we remain superficial. By the end of this programme, you will (hopefully) be equipped to:

- Develop and maintain a professional-grade code base
- Work collaboratively with other data science colleagues on the same code base

Course objectives: What you will achieve

This does not mean that we remain superficial. By the end of this programme, you will (hopefully) be equipped to:

- Develop and maintain a professional-grade code base
- Work collaboratively with other data science colleagues on the same code base
- Test out new ideas and deepen understanding of complex concepts by coding them up from scratch and running experiments

Course objectives: What you will achieve

This does not mean that we remain superficial. By the end of this programme, you will (hopefully) be equipped to:

- Develop and maintain a professional-grade code base
- Work collaboratively with other data science colleagues on the same code base
- Test out new ideas and deepen understanding of complex concepts by coding them up from scratch and running experiments
- Create a data science portfolio on GitHub

Course objectives: What you will achieve

This does not mean that we remain superficial. By the end of this programme, you will (hopefully) be equipped to:

- Develop and maintain a professional-grade code base
- Work collaboratively with other data science colleagues on the same code base
- Test out new ideas and deepen understanding of complex concepts by coding them up from scratch and running experiments
- Create a data science portfolio on GitHub
- Successfully complete data science challenges in job interviews

Part I - The setup

- **Lecture 1: The tech setup**
- Lecture 2: Data wrangling
- Lecture 3: Data retrieval
- Lecture 4: Visualisations

Part II - Modelling

- Lecture 5: Feature engineering
- Lecture 6: Statistical modelling
- Lecture 7: Statistical modelling
- Lecture 8: Model evaluation and interpretability
- Lecture 9: Model tracking and reproducibility

Organisational things

Organisational things

- Nine two-hour lectures: Saturday 9am - 11am
 - All lectures will be online
 - Lectures 1-4 taught by Dr. Adrian Ochs (acro2@cam.ac.uk)
 - Lectures 5-9 taught by Dr. Christian Rörig (cmr61@cam.ac.uk)
- Five two-hour tutorials on week 3, 5, 6, 7, 9 from 11am - 1pm
 - Complete your problem sets in groups
 - TA: Cheuck Ng (cfn24@cam.ac.uk)
- The exam will be some form of a data science project
 - 2 weeks: 4th - 18th December 2025, due at 12 noon UK time
 - You will work alone on this project
- Office hours: Mondays from 9am - 10am.
 - Please sign up for 15 minute slots beforehand via this [link](#)
 - Sending your questions before would also be helpful

Another important thing: breaks

Let's do a **10 minute** break at 9:50 am (please do remind me).

Let's get started - Table of contents for today

3. Why Economists are good Data Scientists

4. What do Data Scientists do?

5. What Data Scientists work with - the tech stack

Python

Jupyter Notebooks

IDEs: Visual Studio Code

Package manager & virtual environments

6. The Data Science code base

Problems of Jupyter Notebook code

How to write modular code

Writing functions and importing them

Orchestrating your modules

The plea for clean code

Recap

7. How Data Scientists collaborate

Git

GitHub

Why Economists are good Data Scientists

Data models vs. black boxes

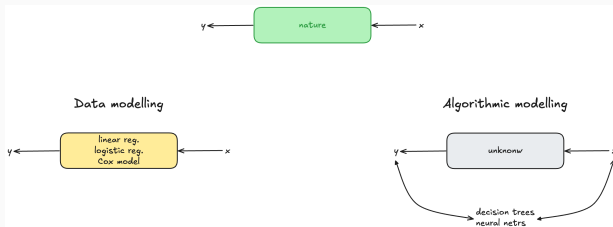


Figure 1: From L. Breiman (2001)

Data models vs. black boxes

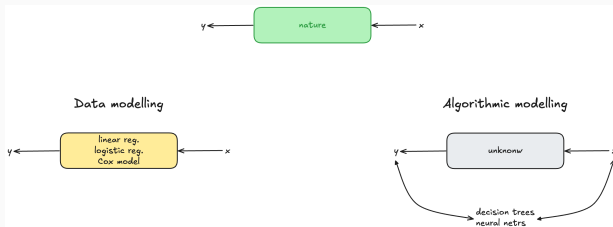


Figure 1: From L. Breiman (2001)

- In Econometrics, we "assume that the data are generated by the following model:..."
- In DS, we mostly work with a black box

Data models vs. black boxes

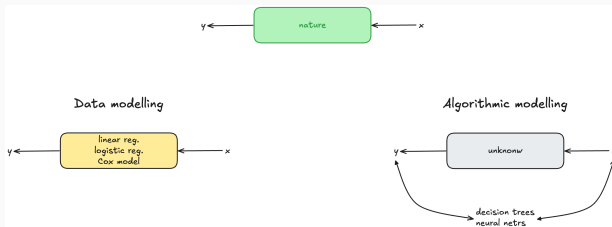


Figure 1: From L. Breiman (2001)

- In Econometrics, we "assume that the data are generated by the following model:..."
- In DS, we mostly work with a black box
- Model valuation in Econometrics: Goodness of fit measure
- Model valuation in DS: Predictive accuracy

Data models vs. black boxes

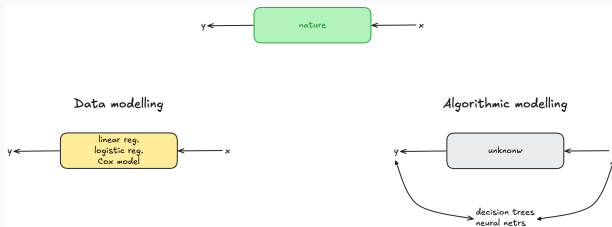


Figure 1: From L. Breiman (2001)

- In Econometrics, we "assume that the data are generated by the following model:..."
- In DS, we mostly work with a black box
- Model valuation in Econometrics: Goodness of fit measure
- Model valuation in DS: Predictive accuracy
- Instead of causal inference, DS mostly cares about prediction
- Reliable, predictive accuracy is more important for a DS than interpretability

Data models vs. black boxes

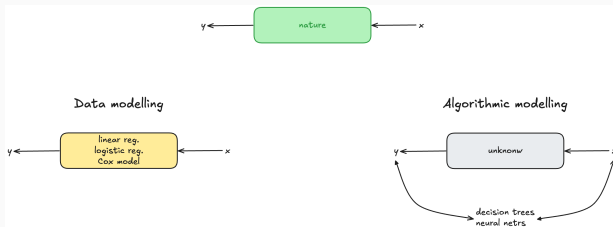


Figure 1: From L. Breiman (2001)

- In Econometrics, we "assume that the data are generated by the following model:..."
- In DS, we mostly work with a black box
- Model valuation in Econometrics: Goodness of fit measure
- Model valuation in DS: Predictive accuracy
- Instead of causal inference, DS mostly cares about prediction
- Reliable, predictive accuracy is more important for a DS than interpretability
- **SWE skills are more important for the black box approach**

So why should Economists do DS?

Your Economics education creates intuition that is essential in ML.

So why should Economists do DS?

Your Economics education creates intuition that is essential in ML.

You have a data set of homes for sale in New York City. Can you use this data set to predict home prices in Albany (New York State)? (Perlich, 2024)

So why should Economists do DS?

Your Economics education creates intuition that is essential in ML.

You have a data set of homes for sale in New York City. Can you use this data set to predict home prices in Albany (New York State)? (Perlich, 2024)

- The data is not relevant. NYC and Albany are too different to be able to make any inference from the NYC data for Albany.
- We say that it is out of sample.
- This is obvious to an economist - we know that the market structure is different in New York City and Albany.

So why should Economists do DS?

Another intuition question:

You are called as an advisor to create an early warning metric to predict crop loss in Colombia. You have several years worth of weather data. What might be the problem? (Perlich, 2024)

So why should Economists do DS?

Another intuition question:

You are called as an advisor to create an early warning metric to predict crop loss in Colombia. You have several years worth of weather data. What might be the problem? (Perlich, 2024)

- Colombia has very homogeneous annual weather conditions → no signal in the data to create a good model.

What do Data Scientists do?

What are the tasks of Data Scientists?

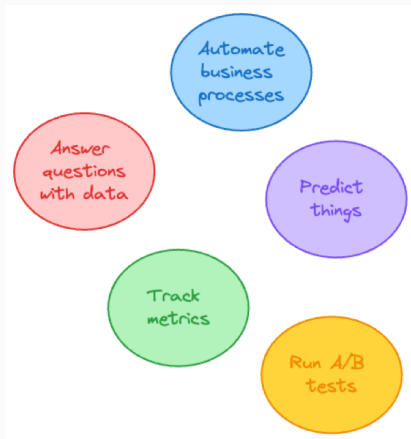


Figure 2: Tasks of DSs

Example tasks of Data Scientists

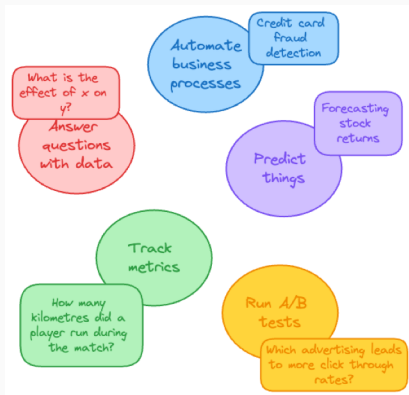


Figure 3: DS task examples

We focus on automation and prediction

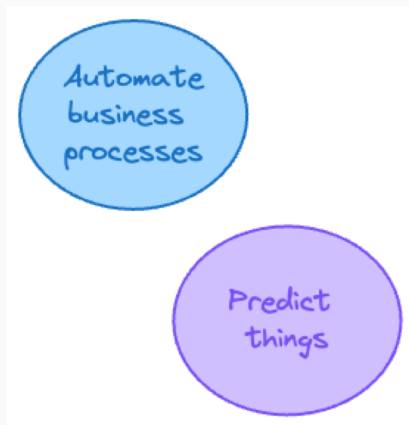


Figure 4: The tasks we focus on.

The Data Science workflow/pipeline

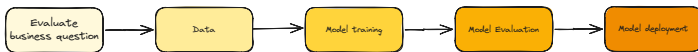


Figure 5: Data science pipeline

The Data Science pipeline is iterative

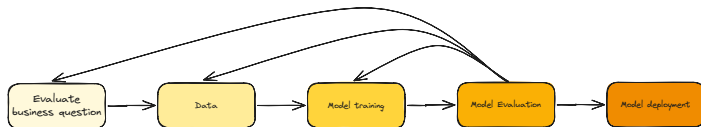


Figure 6: The data science pipeline is iterative

The Data Science pipeline: data

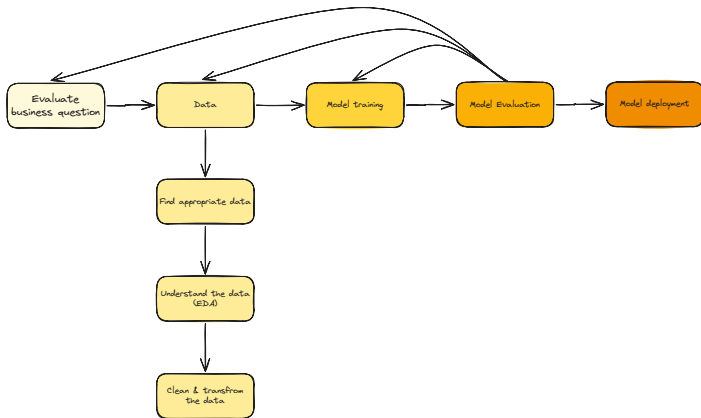


Figure 7: A lot of time is spent on data

DSs spend most their time on data

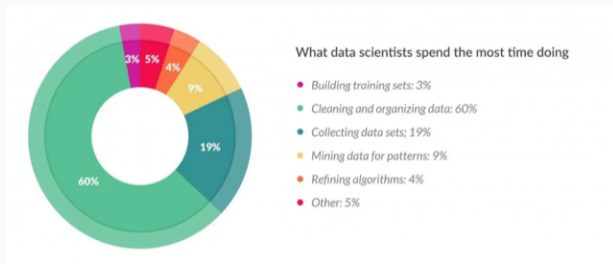


Figure 8: [Forbes article \(2016\)](#)

The Data Science pipeline: modelling

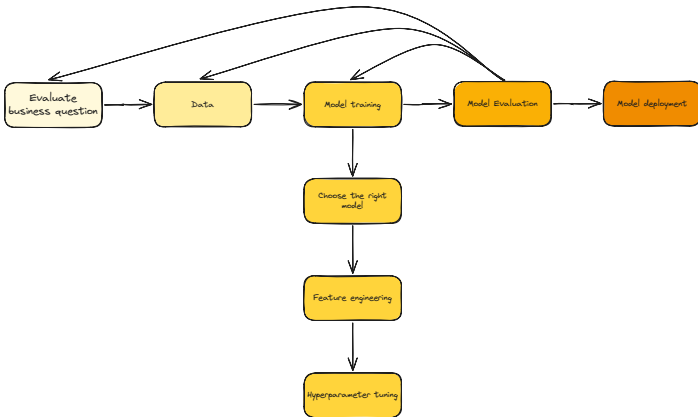


Figure 9: Some time is spent on modelling

The Data Science pipeline in full

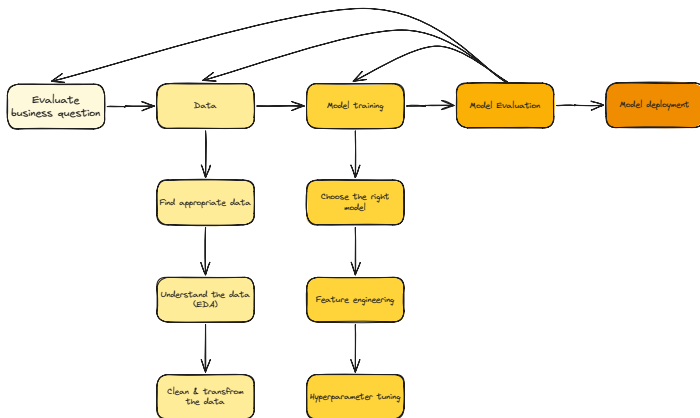


Figure 10: Data science pipeline in full

Your code is also organised following this pipeline

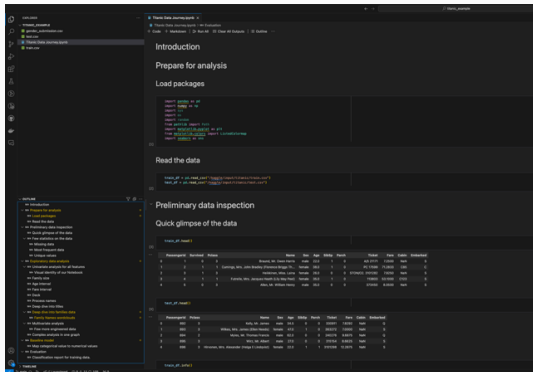
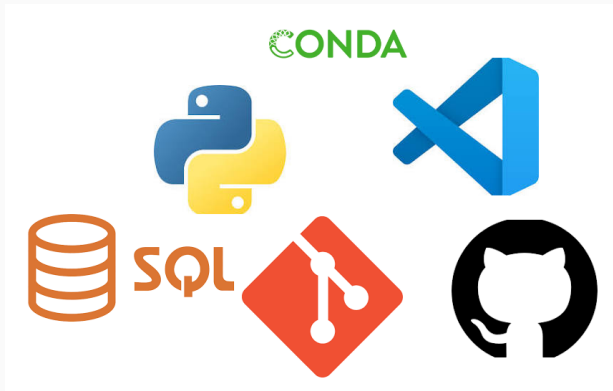


Figure 11: Titanic Kaggle competition example

What Data Scientists work with

- the tech stack**

Our tech stack



The line between DS and SWE should soon become blurry

- Data scientists are involved at every step of the pipeline

The line between DS and SWE should soon become blurry

- Data scientists are involved at every step of the pipeline
- The more software engineering (SWE) skills you have, the more effective you are
- Companies value good SWE practices

The line between DS and SWE should soon become blurry

- Data scientists are involved at every step of the pipeline
- The more software engineering (SWE) skills you have, the more effective you are
- Companies value good SWE practices
- (I think) SWE skills can set you apart in Economics research

The line between DS and SWE should soon become blurry

- Data scientists are involved at every step of the pipeline
- The more software engineering (SWE) skills you have, the more effective you are
- Companies value good SWE practices
- (I think) SWE skills can set you apart in Economics research
- Well-written and organised code is always better

The line between DS and SWE should soon become blurry

- Data scientists are involved at every step of the pipeline
- The more software engineering (SWE) skills you have, the more effective you are
- Companies value good SWE practices
- (I think) SWE skills can set you apart in Economics research
- Well-written and organised code is always better

→ This course emphasises SWE skills and best practices.

Python

We use Python because it is very popular

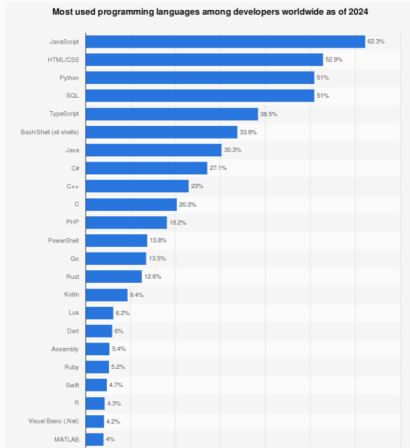


Figure 12: Most used languages 2024 ([Statista](#))

Python has many other advantages

Why not use R, Matlab, or Stata (languages we used to love in Economics)?

- **Machine learning:** Python is the prevailing standard in the field of machine learning
- **Learning curve:** Python is a high-level programming language like R, Matlab, or Stata but:
 - **Versatility:** General-purpose language, beyond just data analysis
 - **Larger ecosystem:** Vast library collection for various purposes
 - **Software development:** Allows you to build complex applications or integrate data analysis and automation processes into larger systems

But it is harder to just get going compared to R, or Stata

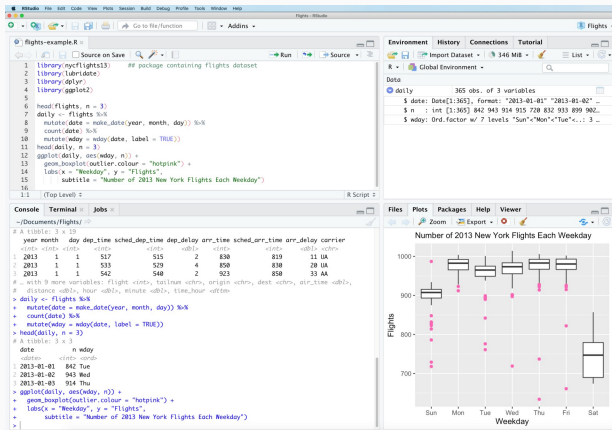


Figure 13: Wikipedia - R Studio

There are many ways to write and execute Python code

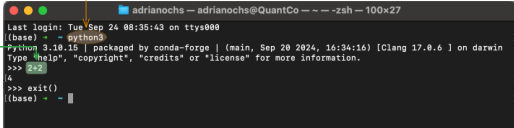
Python is both a programming **language** and a **program**:

- **As a language:** Python is a set of syntax rules and semantics used to write code.
- **As a program:** The Python **interpreter** is software that reads and executes Python code. When you install Python on your computer, you are installing this interpreter program.

Writing and executing Python code from the terminal

Python interpreter

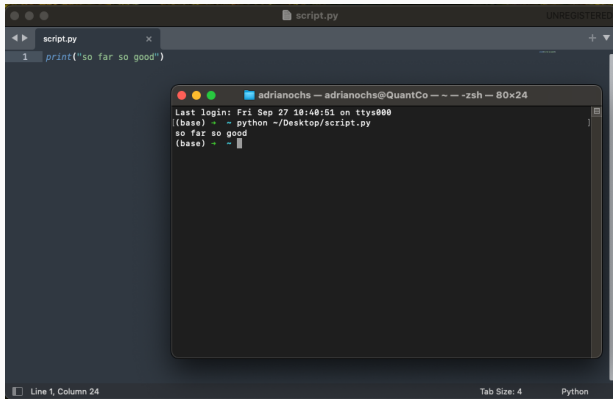
Python syntax



The image shows a terminal window with a dark background. At the top, the window title is 'adrianochs — adrianochs@QuantCo — -zsh — 100x27'. The terminal content shows the user logging in, then typing 'python3' which shows the Python version '3.10.15' and packaging information. The user then enters '>>> 2+2' and the interpreter returns '4'. Finally, the user enters '>>> exit()' and the prompt returns to the shell '((base) ~)'. An orange arrow points from the text 'Python interpreter' to the 'python3' command. A green arrow points from the text 'Python syntax' to the '2+2' expression.

```
Last login: Tue Sep 24 08:35:43 on ttys000
((base) ~) % python3
Python 3.10.15 | packaged by conda-forge | (main, Sep 20 2024, 16:34:16) [Clang 17.0.6 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> 2+2
4
>>> exit()
((base) ~) %
```

Executing a Python script from the terminal



The image shows a code editor window with a tab titled 'script.py'. The code in the editor is a single line: `1 print("so far so good")`. Below the editor, a terminal window is open, showing the command `python ~/Desktop/script.py` being executed. The terminal output is `so far so good`. The terminal window title is `adrianochs — adrianochs@QuantCo — --zsh — 80x24`. The terminal also shows the last login time: `Last login: Fri Sep 27 18:40:51 on ttys000`. The terminal prompt is `(base) -`. The code editor status bar at the bottom indicates `Line 1, Column 24`, `Tab Size: 4`, and `Python`.

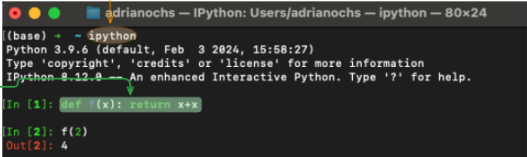
```
script.py
1 print("so far so good")
```

```
adrianochs — adrianochs@QuantCo — --zsh — 80x24
Last login: Fri Sep 27 18:40:51 on ttys000
(base) - python ~/Desktop/script.py
so far so good
(base) -
```

Line 1, Column 24 Tab Size: 4 Python

A different, interactive, interpreter

Interactive python interpreter



The screenshot shows a terminal window titled "adrianochs — IPython: Users/adrianochs — ipython — 80x24". The prompt is "(base) + ~ ipython". The output shows the Python version "Python 3.9.6 (default, Feb 3 2024, 15:58:27)" and the IPython version "IPython 8.12.0" with a message "An enhanced Interactive Python. Type '?' for help." Below this, the user enters "[In [1]: def f(x): return x+x]" and the prompt changes to "[In [2]: f(2)]". The output is "Out[2]: 4".

```
((base) + ~ ipython
Python 3.9.6 (default, Feb 3 2024, 15:58:27)
Type 'copyright', 'credits' or 'license' for more information
IPython 8.12.0 -- An enhanced Interactive Python. Type '?' for help.

[In [1]: def f(x): return x+x

[In [2]: f(2)
Out[2]: 4
```

Python syntax

- This interpreter is used for Jupyter Notebooks.

Jupyter Notebooks

Jupyter notebooks can help you write and execute Python code

Jupyter example Last Checkpoint: 15 minutes ago (unsaved changes)

File Edit View Insert Cell Kernel Help Not Trusted zero_to_hero

Intro

A nice Jupyter Notebook

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
```

```
In [2]: def f(x):
return x*x
```

```
In [3]: y = f(2)
```

```
In [4]: y == 4
Out[4]: True
```

```
In [11]: # Generate sample data
days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
temperatures = np.random.randint(15, 30, 7) # Random temperatures between 15°C and 30°C

# Create a DataFrame
df = pd.DataFrame({'Day': days, 'Temperature': temperatures})

# Create the plot
sns.Lineplot(x='Day', y='Temperature', data=df, marker='o')

Out[11]: <Axes: xlabel='Day', ylabel='Temperature'>
```

Day	Temperature
Mon	28
Tue	26
Wed	19
Thu	28
Fri	22
Sat	18
Sun	21

In []:

Unpopular opinion? Jupyter Notebooks are the McDonald's of Code

You can go to McDonald's and order a salad, but you won't. Same as with notebooks, **you can write NASA-production-grade software in a notebook, but most likely you will not.** Notebooks make you lazy and encourage bad practices. ([Blog post](#))

Unpopular opinion? Jupyter Notebooks are the McDonald's of Code

You can go to McDonald's and order a salad, but you won't. Same as with notebooks, **you can write NASA-production-grade software in a notebook, but most likely you will not.** Notebooks make you lazy and encourage bad practices. ([Blog post](#))

Some of the bad things (there are more later):

- You do not see your code base
- You cannot easily jump to your function definition
- Tab completion is not great
- You cannot work easily with the terminal

Unpopular opinion? Jupyter Notebooks are the McDonald's of Code

You can go to McDonald's and order a salad, but you won't. Same as with notebooks, **you can write NASA-production-grade software in a notebook, but most likely you will not.** Notebooks make you lazy and encourage bad practices. ([Blog post](#))

Some of the bad things (there are more later):

- You do not see your code base
- You cannot easily jump to your function definition
- Tab completion is not great
- You cannot work easily with the terminal

NB: Jupyter lab solves some of these issues, but not as extensively as another solution → **we use IDEs.**

IDEs: Visual Studio Code

An IDE is a software providing many tools for software development

Most IDEs (Integrated Development Environment) include:

- A code editor with features like syntax highlighting and auto-completion
- A debugger
- Version control integration
- Data bank integration
- Code linters

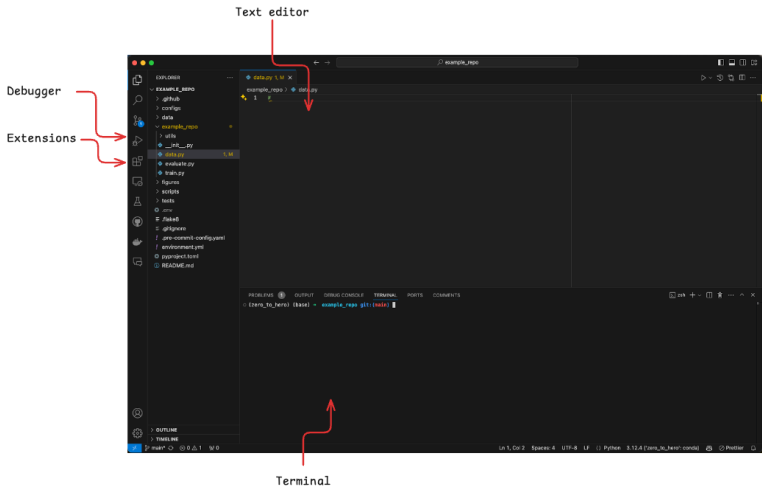
We work with VS Code but there are many IDEs

Worldwide, Aug 2024 :

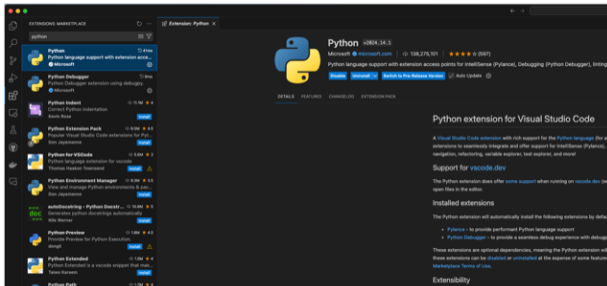
Rank	Change	IDE	Share	1-year trend
1		Visual Studio	27.24 %	-0.8 %
2		Visual Studio Code	14.03 %	+0.1 %
3		Eclipse	11.94 %	+0.6 %
4		pyCharm	10.83 %	+1.6 %
5		Android Studio	9.94 %	+0.8 %
6		IntelliJ	7.89 %	+0.4 %
7		NetBeans	3.9 %	-0.2 %
8		Xcode	3.02 %	-0.1 %
9	↑↑	RStudio	2.81 %	-0.1 %
10	↓	Sublime Text	2.51 %	-0.5 %
11	↓	Atom	2.14 %	-0.9 %
12		Code::Blocks	1.49 %	-0.1 %
13		Vim	0.87 %	-0.0 %
14		PhpStorm	0.4 %	-0.0 %
15		Qt Creator	0.3 %	+0.0 %
16	↑	Emacs	0.19 %	-0.0 %

Top IDE index (how often searched with Google)

This is how VS Code looks



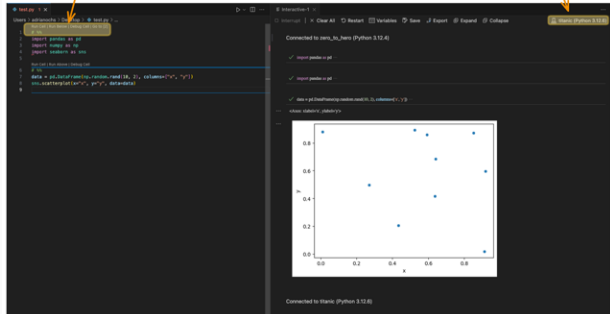
Install the Python extension in VS Code



Python extension let's you work interactively with your code

%%
let's you run code cells

choose your Python interpreter
also in the interactive window

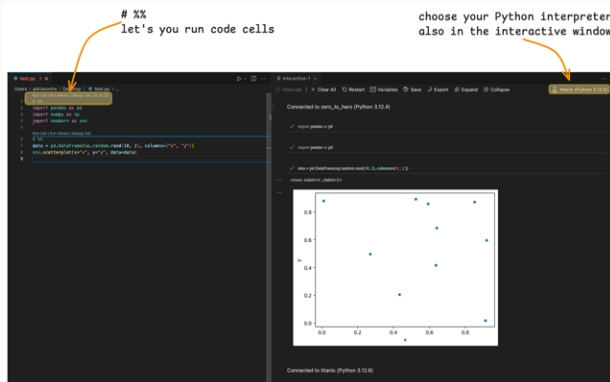


The screenshot displays a Jupyter Notebook interface. On the left, a code cell contains the following Python code:

```
1 import pandas as pd
2 import numpy as np
3 import random as rnd
4
5 # %%
6 data = pd.DataFrame(np.random.randn(10, 2), columns=['x', 'y'])
7 plt.scatter(data['x'], data['y'])
8
```

On the right, the interactive window shows the execution output, including a scatter plot of the data. The plot has 'x' on the horizontal axis and 'y' on the vertical axis, with data points scattered across the range from 0.0 to 1.0 on both axes.

Python extension let's you work interactively with your code



- There are many useful extensions (GitLens, Co-Pilot, Jupyter, ...)
- But use sparingly

Package manager & virtual environments

Packages are collections of related Python code (including functions, classes, and more) that are organised so that they can be easily reused and shared.

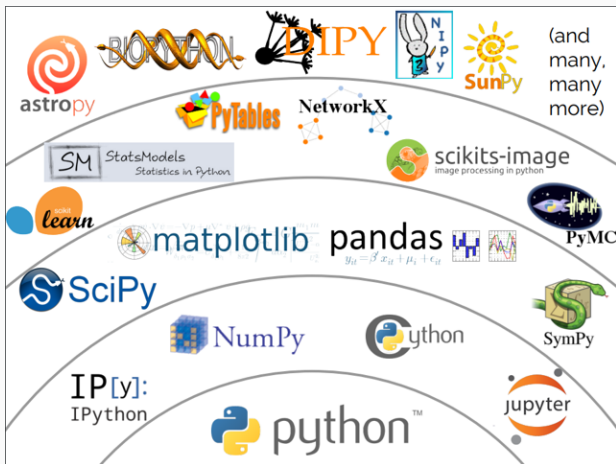
Many built-in packages are already included in Python:

- **datetime:** Working with dates and time
- **re:** Regular expressions

Other packages can be downloaded and installed, e.g.

- **numpy:** Numerical Python
- **pandas:** Data analysis
- **sklearn:** Machine learning

Learn concepts rather than packages



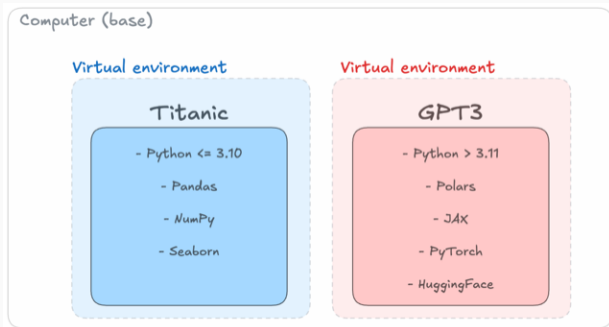
Python stack in 2015 (no PyTorch; Polars vs. Pandas) ([source](#))

We use Conda as a package manager to download and update packages

1. Scope:
 - Pip: Primarily for Python packages
 - Conda: Manages packages for any language, not just Python
2. Default installation:
 - Pip: Comes pre-installed with Python
 - Conda: Requires separate installation of Anaconda or Miniconda
3. Package sources:
 - Pip: Installs from Python Package Index (PyPI)
 - Conda: Uses its own package repository (Anaconda repository)
4. Binary packages & Dependency resolution:
 - Pip: Typically installs source packages that need compilation
 - Conda: Often provides pre-built binary packages and better at resolving complex dependencies → **faster**
5. Environment management:
 - Pip: Requires additional tools (like virtualenv)
 - Conda: Built-in environment management with useful data science packages pre-installed

We also use Conda for virtual environment management

Virtual environments allow you to cleanly separate the specific Python interpreters and set of packages you need for each of your projects.



Why you should use virtual environments

The benefits of working with virtual environments:

- Isolate project dependencies to avoid conflicts between different projects
- Ensure reproducibility
- Facilitate easy collaboration

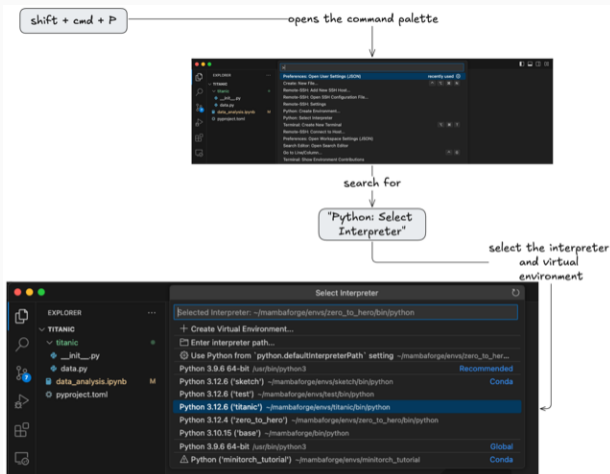
With Conda, creating VEs is easy

In the terminal:

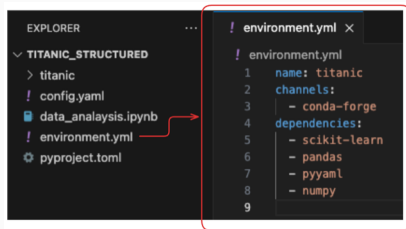
```
1  conda create --name titanic # create environment
2  conda activate titanic # activate environment
3  conda install numpy # install the numpy
4
```

Choose your environment and Python interpreter in VS Code

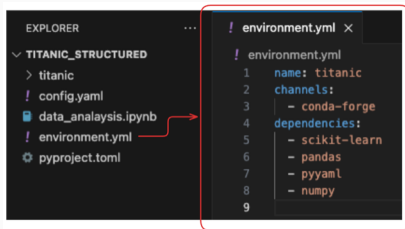
To run Python code in VS Code, you have to choose an interpreter, which is within your virtual environment:



Environment yaml lets you easily share your virtual environment



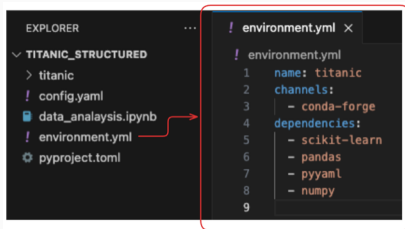
Environment yaml lets you easily share your virtual environment



Install the environment with the packages specified by going to the root of your code directory in the terminal and:

```
1 cd path_to_repo
2 conda env create -f environment.yaml
3 conda activate titanic # activate environment
4
```

Environment yaml lets you easily share your virtual environment



Install the environment with the packages specified by going to the root of your code directory in the terminal and:

```
1  cd path_to_repo
2  conda env create -f environment.yaml
3  conda activate titanic # activate environment
4
```

→ Do you see how this can be useful for reproducibility and collaboration?

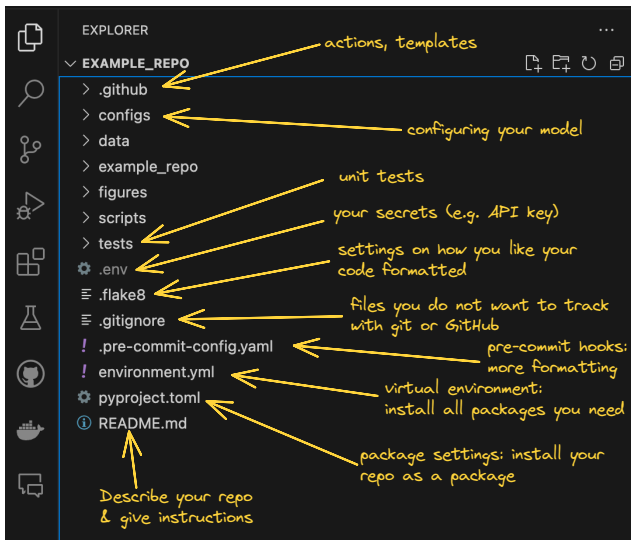
Let's:

- Look at a repo on GitHub
- Clone the repo locally
- Install the virtual environment
- Setup the python interpreter in VS Code

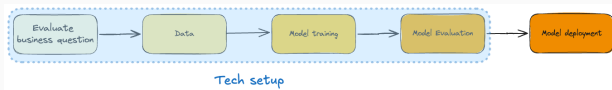
GitHub Repo

The Data Science code base

A DS repo can seem complicated - but you will know all this by the end of this course

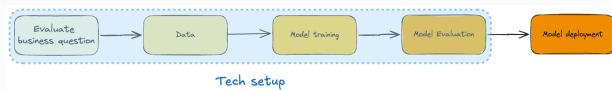


A note on teaching philosophy



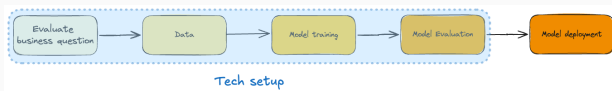
- Each session is a deep dive on a particular part of the pipeline

A note on teaching philosophy



- Each session is a deep dive on a particular part of the pipeline
- Today, we dive deep on the tech and repo setup

A note on teaching philosophy



- Each session is a deep dive on a particular part of the pipeline
- Today, we dive deep on the tech and repo setup
- We add more setup topics throughout the sessions until we arrive at a repo that looks like the previous slide

Problems of Jupyter Notebook code

Remember the unpopular opinion - Jupyter Notebooks are not great to create these code bases

You can go to McDonalds and order a salad, but you won't. Same with notebooks, you can write NASA-production-grade software in a notebook, but most likely you will not. Notebooks make you lazy and encourage bad practices.

Markdown and inline plots are nice, but...

- **State** is problematic; the order of execution matters
- **Writing everything in one file** makes the script super long and confusing
- Notebooks don't go well with **PR reviews**
- Many more issues described in this [talk](#)

Remember the unpopular opinion - Jupyter Notebooks are not great to create these code bases

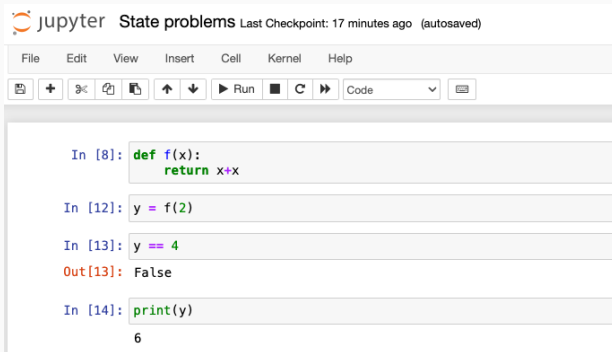
You can go to McDonalds and order a salad, but you won't. Same with notebooks, you can write NASA-production-grade software in a notebook, but most likely you will not. Notebooks make you lazy and encourage bad practices.

Markdown and inline plots are nice, but...

- **State** is problematic; the order of execution matters
- **Writing everything in one file** makes the script super long and confusing
- Notebooks don't go well with **PR reviews**
- Many more issues described in this [talk](#)

→ **These are all problems that are fixed with a proper setup.**

The state problem



The image shows a Jupyter Notebook interface with the title "State problems" and a status bar indicating "Last Checkpoint: 17 minutes ago (autosaved)". The interface includes a menu bar with "File", "Edit", "View", "Insert", "Cell", "Kernel", and "Help". Below the menu bar is a toolbar with icons for saving, adding cells, undo, redo, copy, paste, and running code. The main area displays a series of code cells and their outputs:

```
In [8]: def f(x):  
        return x+x  
  
In [12]: y = f(2)  
  
In [13]: y == 4  
Out[13]: False  
  
In [14]: print(y)  
6
```

Endless scripts, no package versions, and paths that do not work

The whole analysis in one file - happy scrolling 😊

But which versions?

This path won't work on my computer

Introduction

Prepare for analysis

Load packages

```
In [1]: import pandas as pd
import numpy as np
import sys
import os
import random
from pathlib import Path
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import seaborn as sns
```

Read the data

```
In [2]: train_df = pd.read_csv("../kaggle/input/titanic/train.csv")
test_df = pd.read_csv("../kaggle/input/titanic/test.csv")
```

Preliminary data inspection

Quick glimpse of the data

```
In [3]: train_df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 2151	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Hekkinen, Miss. Laina	female	26.0	0	0	STON/O2. 331282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

An analysis I do not want to run all the time

Family Names wordclouds



```
In [74]: from wordcloud import WordCloud, STOPWORDS
stopwords = set(STOPWORDS)

def show_wordcloud(data, mask=None, title=""):
    text = " ".join(t for t in data.dropna())
    stopwords = set(STOPWORDS)
    stopwords.update(["t", "co", "https", "amp", "u", "Comment", "text", "attr", "object"])
    wordcloud = WordCloud(stopwords=stopwords, scale=4, max_font_size=50, max_words=500, mask=mask, background_color="white", colormap=cm.cmap_custom).generate(text)
    fig = plt.figure(1, figsize=(12, 12))
    plt.axis('off')
    fig.suptitle(title, fontsize=14)
    fig.subplots_adjust(top=2.3)
    plt.imshow(wordcloud, interpolation='bilinear')
    plt.show()
```

```
In [75]: show_wordcloud(all_df["Family Name"], title="Family Names on Titanic")
```



Family Names on Titanic

Hard coded parameters!

Baseline model

```
In [97]: from sklearn.model_selection import train_test_split
        from sklearn import metrics
        from sklearn.metrics import roc_auc_score
        from sklearn.ensemble import RandomForestClassifier
```

Map categorical value to numerical values

```
In [98]: for dataset in [train_df, test_df]:
        dataset['Sex'] = dataset['Sex'].map( {'female': 1, 'male': 0} ).astype(int)
```

Create train-validation split.

```
In [99]: VALID_SIZE = 0.2
        train, valid = train_test_split(train_df, test_size=VALID_SIZE, random_state=42, shuffle=True)
```

Define predictor features and target feature.

```
In [100]: predictors = ["Sex", "Pclass"]
        target = 'Survived'
```

Define the training and validation data and labels.

```
In [101]: train_X = train[predictors]
        train_Y = train[target].values
        valid_X = valid[predictors]
        valid_Y = valid[target].values
```

Initialize the classification algorithm.

```
In [102]: clf = RandomForestClassifier(n_jobs=-1,
        random_state=42,
        criterion="gini",
        n_estimators=100,
        verbose=False)
```

Hard coded parameters 🤖

Fit the classifier with the training data.

```
In [103]: clf.fit(train_X, train_Y)
```

```
Out [103]: RandomForestClassifier(n_jobs=-1, random_state=42, verbose=False)
```

Predict the train data (to check the training classification error).

```
In [104]: preds_tr = clf.predict(train_X)
```

Summarising the problems we have observed

1. The state problem: the order of execution is easy to mess up
2. Notebooks become very long and hard to navigate
3. Code that will have to run often is mixed with one-time analyses
4. Notebooks promote bad habits:
 - Not providing package versions
 - Hard coded parameters

How to write modular code

We want modular code that's easy to understand, replicate, and build on

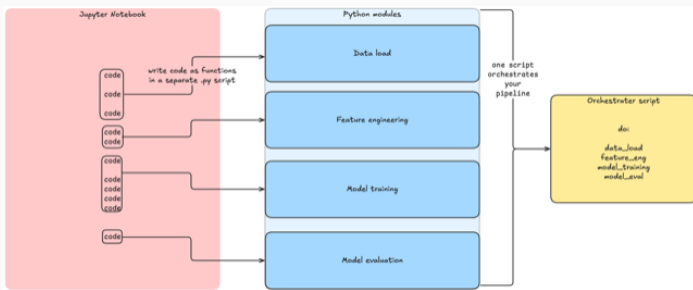
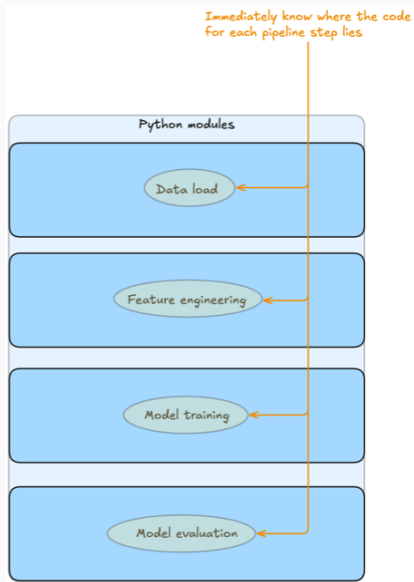
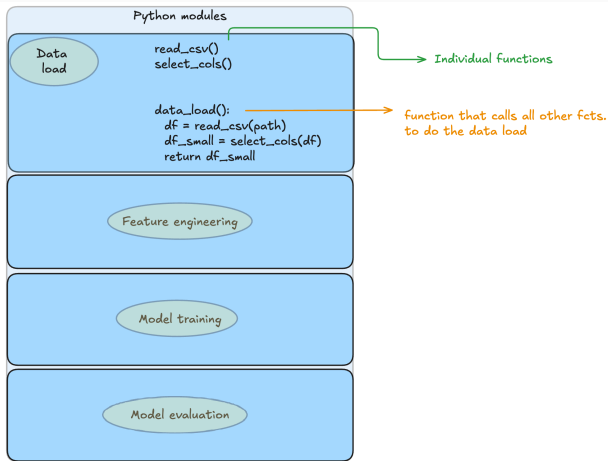


Figure 14: Transforming a notebook into modular code that is executed by another *orchestration* script.

Modular code is easy to read



Modular code follows a clear structure



Turning titanic notebook into modular code

modularise
in data.py

```
import pandas as pd
train_data = pd.read_csv('train.csv')
test_data = pd.read_csv('test.csv')
train_data.head()
test_data.head()
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0
2	1	1	Captain, Mrs. John Bradley Thompson	female	38.0	1	0
3	1	3	Corey, Mr. John	male	33.0	0	0

Orchestrate your training pipeline

```
from titanic.data import load_data
from titanic.feature_engineering import feature_engineering
from titanic.model import training_model

def training_pipeline():
    df = load_data()
    df = feature_engineering(df)
    model_results = training_model()
    return model_results
```

Looking inside the module: model.py

Jupyter Notebook

Create train-validation split.

```
In [99]: VALID_SIZE = 0.2
         train, valid = train_test_split(train_df, test_size=
```

Define predictor features and target feature.

```
In [100]: predictors = ["Sex", "Pclass"]
          target = 'Survived'
```

Define the training and validation data and labels.

```
In [101]: train_X = train[predictors]
          train_Y = train[target].values
          valid_X = valid[predictors]
          valid_Y = valid[target].values
```

Initialize the classification algorithm.

```
In [102]: clf = RandomForestClassifier(n_jobs=-1,
                                     random_state=42,
                                     criterion="gini",
                                     n_estimators=100,
                                     verbose=False)
```

Fit the classifier with the training data.

```
In [103]: clf.fit(train_X, train_Y)
```

Modular code in .py files

individual function

[illegible]

Function that does the whole training by also calling other functions.
Will be called in the pipeline orchestration script.

Looking inside the module: model.py

Jupyter Notebook

Create train-validation split.

```
In [99]: VALID_SIZE = 0.2
train, valid = train_test_split(train_df, test_size=
```

Define predictor features and target feature.

```
In [100]: predictors = ["Sex", "Pclass"]
          target = 'Survived'
```

Define the training and validation data and labels.

```
In [101]: train_X = train[predictors]
          train_Y = train[target].values
          valid_X = valid[predictors]
          valid_Y = valid[target].values
```

Initialize the classification algorithm.

```
In [102]: clf = RandomForestClassifier(n_jobs=-1,
                                     random_state=42,
                                     criterion="gini",
                                     n_estimators=100,
                                     verbose=False)
```

Fit the classifier with the training data.

```
In [103]: clf.fit(train_X, train_Y)
```

Modular code in .py files

individual function

The screenshot shows a Jupyter Notebook interface. On the left is a file explorer with a tree view containing files like `titanic_train.py`, `config.py`, `data_loader.py`, `data_preprocessor.py`, `environment.yml`, and `pyproject.toml`. The main area displays Python code for training a model. The code includes imports for `sklearn` and `tensorflow`, and defines two functions: `train_test_splitting` and `training_model`. The `train_test_splitting` function takes a dataset and splits it into training and testing sets. The `training_model` function loads a configuration, sets up the data path, and trains a `RandomForestClassifier` model using the training data.

```

1 from os import path
2
3 from sklearn.model_selection import train_test_split
4 from sklearn.ensemble import RandomForestClassifier
5 import pandas as pd
6 from titanic.config import load_config
7 from pathlib import Path
8
9
10 def train_test_splitting
11     df: pd.DataFrame, targets: str, predictors: list(str), split: int, random_state: int
12     i = MultiIndex.from_arrays([df.columns, df.columns])
13     x = df[predictors]
14     y = df[targets]
15     train, valid = train_test_split(x, y, test_size=split, random_state=random_state)
16     train_X = train[predictors].values
17     train_y = train[targets].values
18     valid_X = valid[predictors].values
19     valid_y = valid[targets].values
20     return train_X, train_y, valid_X, valid_y
21
22
23 def training_model():
24     config = load_config()
25     path_to_clean_data = Path(config["parent.parent / config/cleaned_data_path"])
26     df = pd.read_csv(path_to_clean_data, dtype=str)
27     train_X, train_y, valid_X, valid_y = train_test_splitting(
28         df,
29         config["model"]["variables"]["target"],
30         config["model"]["variables"]["predictors"],
31         config["train_test_splitting"]["split"],
32         config["train_test_splitting"]["random_state"],
33     )
34
35     clf = RandomForestClassifier()
36     m_estimator = config["model"]["m_estimators"]
37     random_state = config["model"]["random_state"]
38     criterion = config["model"]["criterion"]
39
40     clf.fit(train_X, train_y)
41     return clf

```

Function that does the whole training by also calling other functions.
Will be called in the pipeline orchestration script.

- Within the modules, we have re-written the Jupyter Notebook code as functions

Writing functions and importing them

Turning code into a function

```
total = data.isnull().sum()
percent = (data.isnull().sum()/data.isnull().count()*100)
tt = pd.concat([total, percent], axis=1, keys=['Total', 'Percent'])
types = []
for col in data.columns:
    dtype = str(data[col].dtype)
    types.append(dtype)
tt['Types'] = types
df_missings_types = np.transpose(tt)
```

rewritten
as
function

```
def missing_data(data):
    """
    This function will return a table containing:
    - the total number of missin values in each column
    - the percentage of missing data in each column.
    - the data type for each column

    Parameters:
    data: a pandas dataframe

    """
    total = data.isnull().sum()
    percent = data.isnull().sum() / data.isnull().count() * 100
    tt = pd.concat([total, percent], axis=1, keys=["Total", "Percent"])
    types = []
    for col in data.columns:
        dtype = str(data[col].dtype)
        types.append(dtype)
    tt["Types"] = types
    return np.transpose(tt)
```

```
missing_data(train_df)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
Total	0	0	0	0	0	177	0	0	0	0	687	2
Percent	0.0	0.0	0.0	0.0	0.0	19.86532	0.0	0.0	0.0	0.0	77.104377	0.224467
Types	int64	int64	int64	object	object	float64	int64	int64	object	float64	object	object

And importing that function in another script



The screenshot shows a Jupyter Notebook interface. On the left, a file explorer sidebar displays a folder named 'titanic' which contains two files: 'data.py' and 'data_analysis.ipynb'. The 'data_analysis.ipynb' file is currently selected. On the right, the notebook's code area shows two input cells. The first cell contains the code 'from titanic.data import missing_data'. The second cell contains the code 'train_missing = missing_data(df_train)' and 'test_missing = missing_data(df_test)'.

```
In [ ]: from titanic.data import missing_data
```

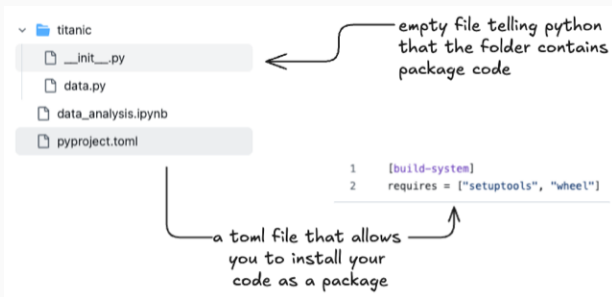
```
In [ ]: train_missing = missing_data(df_train)
        test_missing = missing_data(df_test)
```

Here are all the advantages of functions (that Claude and I could come up with)

1. **Reusability:** You can call functions multiple times without rewriting the same code.
2. **Modularity:** Functions help break down complex problems into smaller, manageable pieces.
3. **Readability:** Well-named functions make code more self-documenting and easier to understand.
4. **Maintainability:** Isolating functionality in functions makes it easier to update or fix specific parts of your code.
5. **Testing:** Functions can be tested independently, simplifying the debugging process.
6. **Abstraction:** Functions hide implementation details, allowing you to focus on high-level logic.
7. **Code organisation:** Functions help structure your code logically.
8. **Collaboration:** Functions make it easier for multiple developers to work on different parts of a project.

I lied a bit: there are three things you have to do to make the code run

1. Add an **init.py** script in the source folder
2. Add a **pyproject.toml** to the root of your directory



I lied a bit: there are three things you have to do to make the code run

3. Install your code as a package

- In the terminal, navigate to your root directory `cd path_to_repo`
- And then type `pip install -e .`

```
(titanic) → titanic git:(main) ✖ pip install -e .
```

Modularity solves some of the Notebook problems

1. The state problem: the order of execution is easy to mess up
2. ~~Notebooks become very long and hard to navigate~~
3. ~~Code that will have to run often is mixed with one-time analyses~~
4. Notebooks promote bad habits:
 - Not providing package versions
 - Hard coded parameters

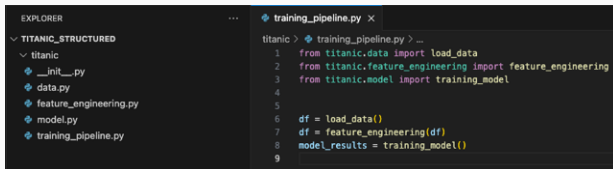
Let's continue what we started before but now:

- Install the repo itself

[GitHub Repo](#)

Orchestrating your modules

The orchestration script



The screenshot shows a code editor interface. On the left, the 'EXPLORER' sidebar displays a file tree for a project named 'TITANIC_STRUCTURED'. Under the 'titanic' folder, the following files are listed: `__init__.py`, `data.py`, `feature_engineering.py`, `model.py`, and `training_pipeline.py`. The `training_pipeline.py` file is selected, and its contents are displayed in the main editor area. The script is a Python file that imports functions from other modules and executes a pipeline.

```
titanic > training_pipeline.py > ...
1  from titanic.data import load_data
2  from titanic.feature_engineering import feature_engineering
3  from titanic.model import training_model
4
5
6  df = load_data()
7  df = feature_engineering(df)
8  model_results = training_model()
9
```

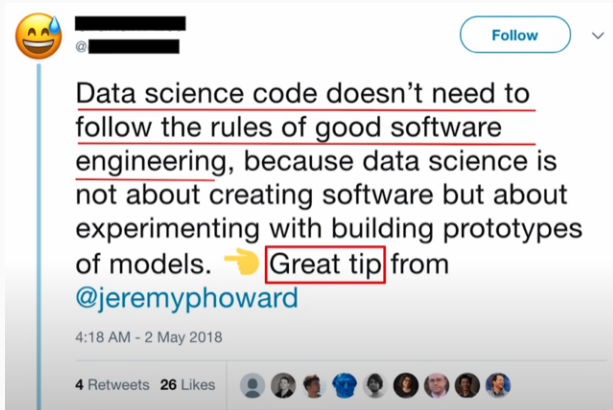
This script orchestrates your pipeline and executes everything in the right order.

Modularity and orchestration script solve the state problem

1. ~~The state problem: the order of execution is easy to mess up~~
2. ~~Notebooks become very long and hard to navigate~~
3. ~~Code that will have to run often is mixed with one-time analyses~~
4. Notebooks promote bad habits:
 - Not providing package versions
 - Hard coded parameters

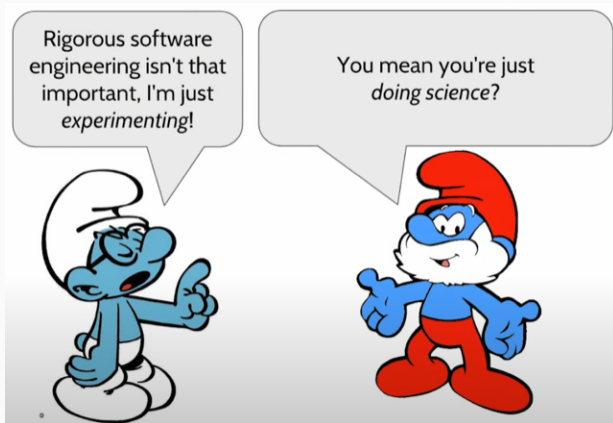
The plea for clean code

Writing clean code, always?



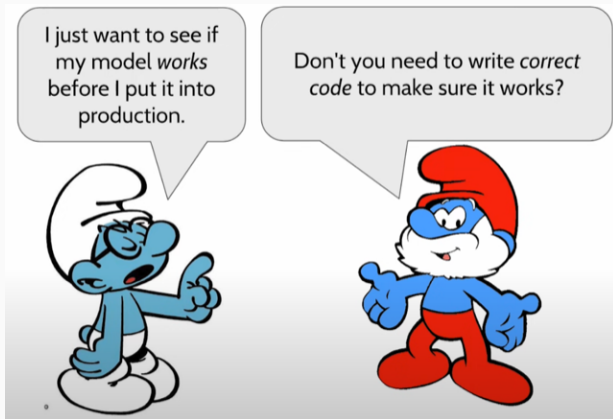
From presentation by [Joel Grus](#)

Experimenting = Doing science



From presentation by [Joel Grus](#)

You should be able to trust your code



From presentation by [Joel Grus](#)

Recap

Our code base/repository is starting to look professional

✓ TITANIC_STRUCTURED

✓ titanic

🔗 `__init__.py`

🔗 `data.py`

🔗 `feature_engineering.py`

🔗 `model.py`

! `environment.yml`

⚙ `pyproject.toml`

🔗 `training_pipeline.py`

Recap: Do you remember what each file does?

```

✓ TITANIC_STRUCTURED
  ✓ titanic
    🌀 __init__.py
    🌀 data.py
    🌀 feature_engineering.py
    🌀 model.py
    ! environment.yml
    ⚙️ pyproject.toml
    🌀 training_pipeline.py

```

What are these files?

✓ TITANIC_STRUCTURED

✓ titanic

🔗 __init__.py

🔗 data.py

🔗 feature_engineering.py

🔗 model.py

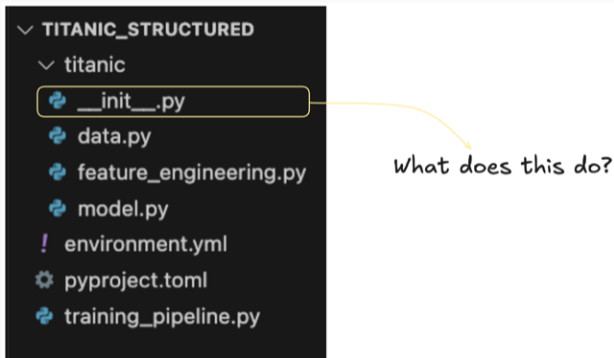
! environment.yml

⚙️ pyproject.toml

🔗 training_pipeline.py

→ Your modules

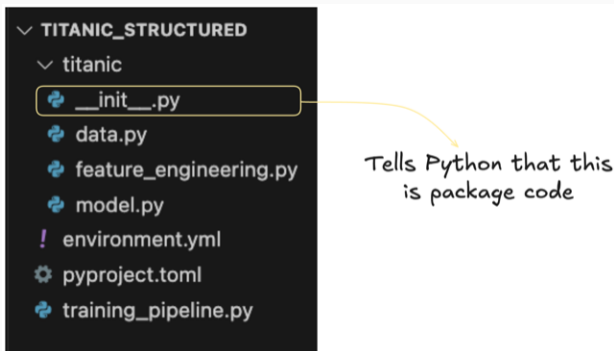
Recap: Do you remember what each file does?



▼ TITANIC_STRUCTURED

- ▼ titanic
 -  `__init__.py`
 -  `data.py`
 -  `feature_engineering.py`
 -  `model.py`
 -  `environment.yml`
 -  `pyproject.toml`
 -  `training_pipeline.py`

What does this do?



File explorer structure:

- ✓ TITANIC_STRUCTURED
 - ✓ titanic
 - `__init__.py`
 - `data.py`
 - `feature_engineering.py`
 - `model.py`
 - ! `environment.yml`
 - ⚙ `pyproject.toml`
 - `training_pipeline.py`

Tells Python that this is package code

Recap: Do you remember what each file does?

▼ TITANIC_STRUCTURED

▼ titanic

🔗 __init__.py

🔗 data.py

🔗 feature_engineering.py

🔗 model.py

! environment.yml

⚙️ pyproject.toml

🔗 training_pipeline.py

what does this do?



Solution

✓ TITANIC_STRUCTURED

✓ titanic

🔗 __init__.py

🔗 data.py

🔗 feature_engineering.py

🔗 model.py

! environment.yml

⚙️ pyproject.toml

🔗 training_pipeline.py

Allows you to install your
code as a package

Recap: Do you remember what each file does?

✓ TITANIC_STRUCTURED

✓ titanic

🔗 __init__.py

🔗 data.py

🔗 feature_engineering.py

🔗 model.py

! environment.yml

⚙️ pyproject.toml

🔗 training_pipeline.py

what does this do?



✓ TITANIC_STRUCTURED

✓ titanic

🔗 __init__.py

🔗 data.py

🔗 feature_engineering.py

🔗 model.py

! environment.yml

⚙️ pyproject.toml

🔗 training_pipeline.py

Orchestration script

Recap: Do you remember what each file does?

✓ TITANIC_STRUCTURED

✓ titanic

🔗 __init__.py

🔗 data.py

🔗 feature_engineering.py

🔗 model.py

! environment.yml

⚙️ pyproject.toml

🔗 training_pipeline.py

What does this
help you with?

Solution

✓ TITANIC_STRUCTURED

✓ titanic

🔗 __init__.py

🔗 data.py

🔗 feature_engineering.py

🔗 model.py

! environment.yml

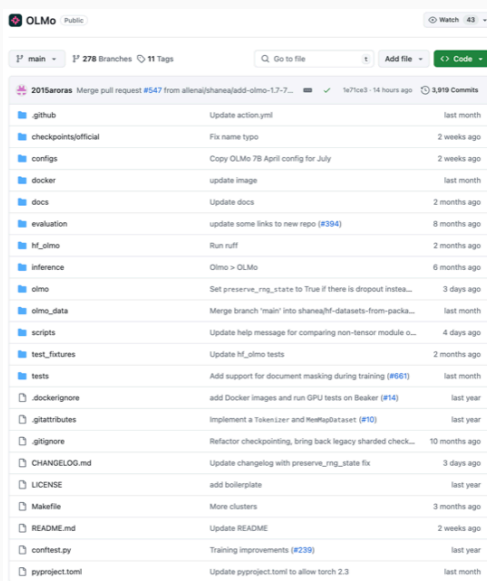
⚙️ pyproject.toml

🔗 training_pipeline.py

Install a virtual env.



We are already close to a real-life SOTA repo: OLMo



The screenshot shows the GitHub repository for OLMo, which is public. It has 278 branches and 11 tags. A recent pull request #547 by 2015aroras is highlighted. The repository contains various files and folders, each with a commit history.

File/Folder	Commit Message	Commit Time
.github	Update action.yml	last month
checkpoints/official	Fix name typo	2 weeks ago
configs	Copy OLMo 7B April config for July	2 weeks ago
docker	update image	last month
docs	Update docs	2 months ago
evaluation	update some links to new repo (#394)	8 months ago
hf_olmo	Run ruff	2 months ago
inference	Olmo > OLMo	6 months ago
olmo	Set preserve_rng_state to True if there is dropout instea...	3 days ago
olmo_data	Merge branch 'main' into shanea/hf-datasets-from-packa...	last month
scripts	Update help message for comparing non-tensor module o...	4 days ago
test_fixtures	Update hf_olmo tests	2 months ago
tests	Add support for document masking during training (#61)	last month
.dockerignore	add Docker images and run GPU tests on Beaker (#14)	last year
.gitattributes	Implement a Tokenizer and MedMapDataset (#10)	last year
.gitignore	Refactor checkpointing, bring back legacy sharded check...	10 months ago
CHANGELOG.md	Update changelog with preserve_rng_state fix	3 days ago
LICENSE	add boilerplate	last year
Makefile	More clusters	3 months ago
README.md	Update README	2 weeks ago
confest.py	Training improvements (#239)	last year
pyproject.toml	Update pyproject.toml to allow torch 2.3	last month

How Data Scientists collaborate

DS is never done alone

- Other than in academia, you will always work with other people on one code base
- This is great!
- You give each other feedback and catch mistakes
- You increase and improve your code base step-by-step

We collaborate with Git and GitHub



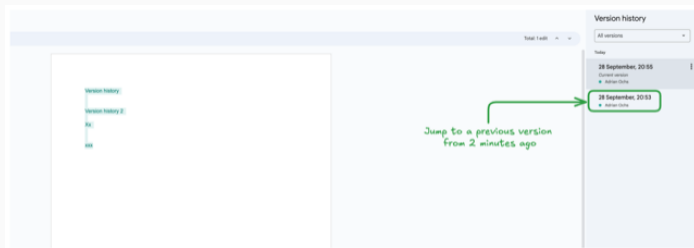
git



- Git is for version control
- GitHub let's you host your repos and collaborate on code

Git

Version control is Word history on steroids



It let's you avoid v1, v2, v3, final, final_final, really_final

```
mycode.py  
mycode_v2.py  
mycode_v2_conference.py  
mycode_v3_BROKEN.py  
mycode_v3_FIXED.py  
mycode_v2+v3.py  
mycode_final.py  
mycode_final_final.py
```

It's also a bit like accepting comments or not

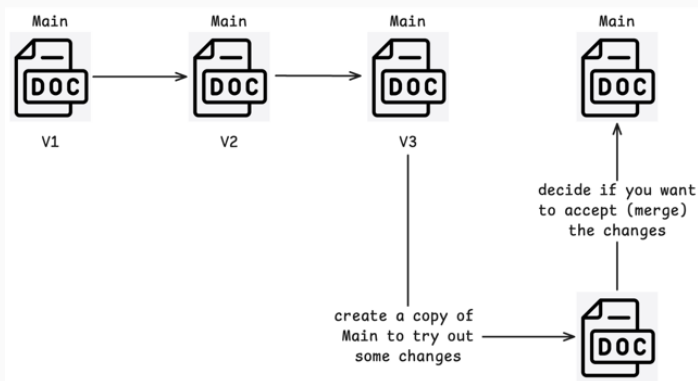


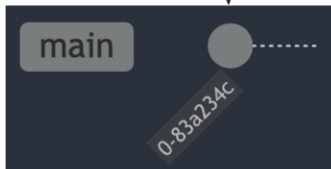
Figure 15: Icon from [here](#)

The typical Git workflow - your first commit

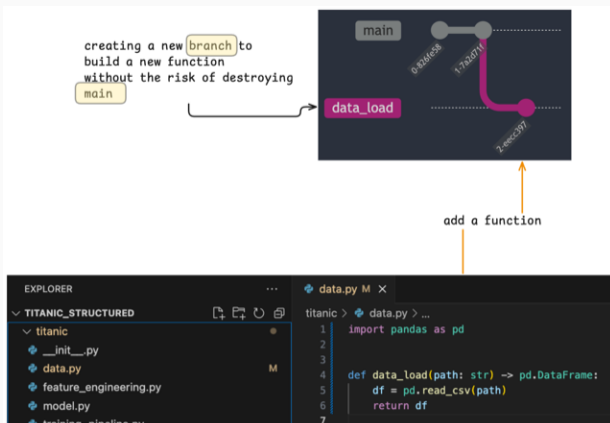
Create a repo with an initial structure and empty modules

```
▼ TITANIC_STRUCTURED
  ▼ titanic
    • __init__.py
    • data.py
    • feature_engineering.py
    • model.py
    • training_pipeline.py
```

save the changes



The typical Git workflow - creating a branch



The typical Git workflow - merge the branch back into main

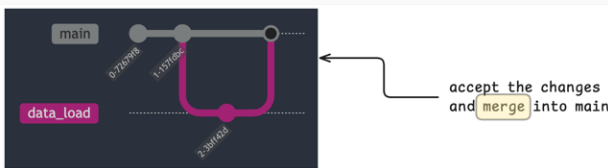


Figure 16: Enter Caption

Git from the terminal - your first commit

```
1  git status  # see all the files you changed
2  git add .   # select all the files
3  git commit -m "start project" # save changes
4
```

Create a repo with an initial structure and empty modules

✓ TITANIC_STRUCTURED

✓ titanic

✚ __init__.py

✚ data.py

✚ feature_engineering.py

✚ model.py

✚ training_pipeline.py

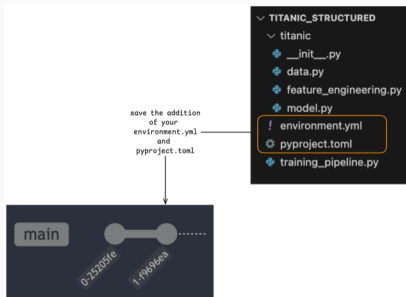
save the changes

main

0-83a234c

Git from the terminal - another commit

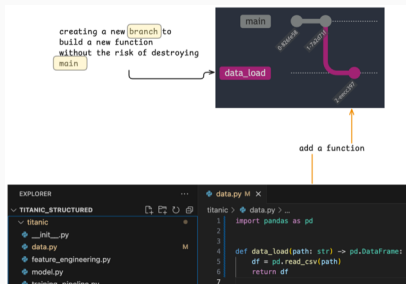
```
1 git add environment.yml # select yaml file
2 git commit -m "create yaml file" # save changes
3
```



The typical Git workflow - creating a branch

(Almost) always create the branch from main:

```
1 git branch data_load # create new branch
2 git checkout data_load # switches branch
3
```



The typical Git workflow - merge the branch back into main

To merge a branch back into main, you first have to go back to main and merge into it:

```
1  git checkout main # back to main branch
2  git merge # merge the changes into main
3
```

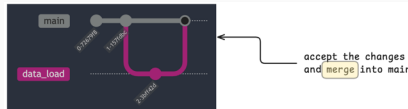


Figure 17: Enter Caption

Things will break - do not panic

Sometimes things get confusing & there might be merge conflicts!

Things will break - do not panic

Sometimes things get confusing & there might be merge conflicts! Don't worry:

- ChatGPT is a great companion to help
- You will get plenty of practical exercises to work with Git
- Just working with Git is a much better way of learning than discussing every detail

GitHub

GitHub let's you store your repo online and collaborate

The screenshot shows the GitHub interface for a repository named 'titanic' by user 'cambridge-FDS'. The repository is private. The main content area displays a file list with columns for file name, type, and last commit. The files listed are 'adrianocha', 'titanic', 'data_analysis.ipynb', and 'pyproject.toml'. Below the file list is a section for adding a README. The right sidebar contains metadata about the repository, including the 'About' section (no description provided), 'Activity' (0 stars, 1 watching, 0 forks), 'Releases' (no releases published), 'Packages' (no packages published), and 'Languages' (Python 57.0%, Jupyter Notebook 43.0%).

cambridge-FDS / titanic 5

Code Issues Pull requests Actions Projects Security Insights Settings

titanic Private Edit file Unwatch 1 Fork Star

main 1 branch 0 tags Go to file Add file Code

File	Type	Last Commit
adrianocha	package infra	9d133d1 - yesterday 2 Commits
titanic	package infra	yesterday
data_analysis.ipynb	ini	yesterday
pyproject.toml	package infra	yesterday

README

Add a README
Add a README with an overview of your project.
[Add a README](#)

About
No description, website, or topics provided.

Activity
0 stars
1 watching
0 forks

Releases
No releases published
[Create a new release](#)

Packages
No packages published
[Publish your first package](#)

Languages

- Python 57.0%
- Jupyter Notebook 43.0%

You can push your changes to GitHub

Once you have saved your changes you can send them (i.e. "push") to GitHub:

```
1  git add .  
2  git commit -m "fix speeling mistake"  
3  git push  
4
```

And pull the changes others made

To update your repo locally with the changes that others made, "pull" them:

```
1  git pull
```

```
2
```

And pull the changes others made

To update your repo locally with the changes that others made, "pull" them:

```
1  git pull
2
```

Note: only pulls the changes for the branch you are on

Working locally and remotely with Git and GitHub

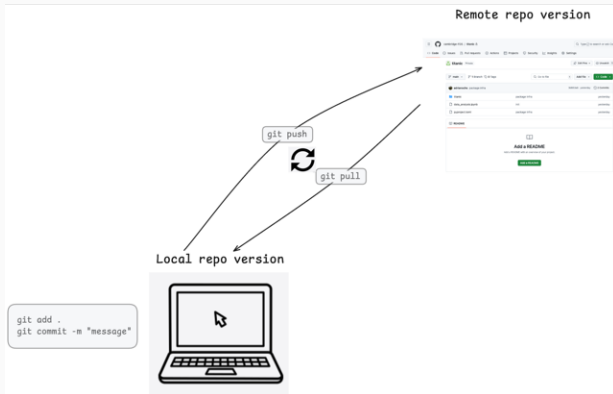
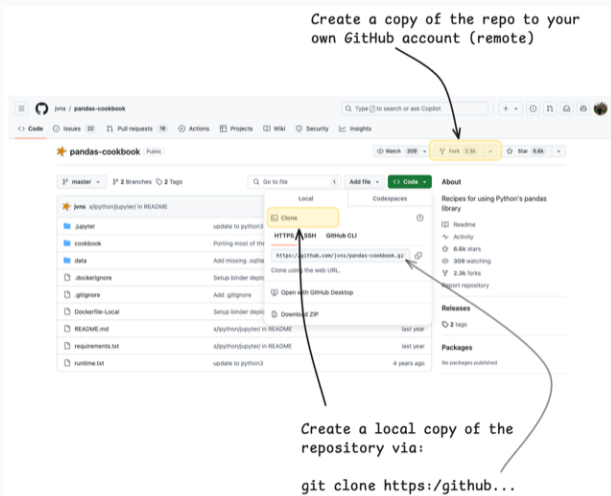


Figure 18: Icon from [here](#).

You can clone or fork the repositories of others

Create a copy of the repo to your own GitHub account (remote)



Create a local copy of the repository via:
git clone https://github...

Cloning for collaboration and forking for own copy

1. Cloning:

- 1.1 Creates a local copy of a repository on your machine.
- 1.2 You can clone any repository you have access to.
- 1.3 Typically used when you want to work on a project locally.
- 1.4 The cloned repo is directly linked to the original remote repository.
- 1.5 If you have write permissions, you can push changes directly to the original repo.

2. Forking:

- 2.1 Creates a copy of a repository in your GitHub (or similar platform) account.
- 2.2 It's a remote copy, not a local one.
- 2.3 Typically used when you want to contribute to a project you don't have write access to.
- 2.4 The forked repo is your own copy, separate from the original.
- 2.5 You can make changes to your fork without affecting the original repo.
- 2.6 Often used as a starting point for pull requests in open-source contributions.

Pull requests to make a change to your code base

The screenshot shows a GitHub pull request page for the repository 'cambridge-FDS / fun_ds'. The pull request title is '[WIP] simple data retrieval example #1'. It was created by 'ChristianRoerig' and aims to merge a commit from the 'data_retrieval' branch into the 'main' branch. The interface includes a top navigation bar with links to Code, Issues, Pull requests (1), Actions, Projects, Security, Insights, and Settings. Below the title, there are statistics: 0 conversations, 1 commit, 0 checks, and 3 files changed. A comment from 'ChristianRoerig' from 3 weeks ago contains a checklist with one item: 'Added a CHANGELOG.rst entry'. Below the comment, the pull request details are shown, including a rule to 'Require approval from specific reviewers before merging' and a green checkmark indicating 'This branch has no conflicts with the base branch'. At the bottom, there is a 'Merge pull request' button and a section for 'Add a comment' with a text input field and a rich text editor toolbar.

cambridge-FDS / fun_ds

Type to search or ask Copilot

< Code Issues Pull requests 1 Actions Projects Security Insights Settings

[WIP] simple data retrieval example #1

Open ChristianRoerig wants to merge 1 commit into main from data_retrieval

Conversation 0 Commits 1 Checks 0 Files changed 3

ChristianRoerig commented 3 weeks ago

Checklist

- ☐ Added a CHANGELOG.rst entry

simple data retrieval example f2fc375

Require approval from specific reviewers before merging
Rulesets ensure specific people approve pull requests before they're merged. Add rule X

☒ This branch has no conflicts with the base branch
Merging can be performed automatically.

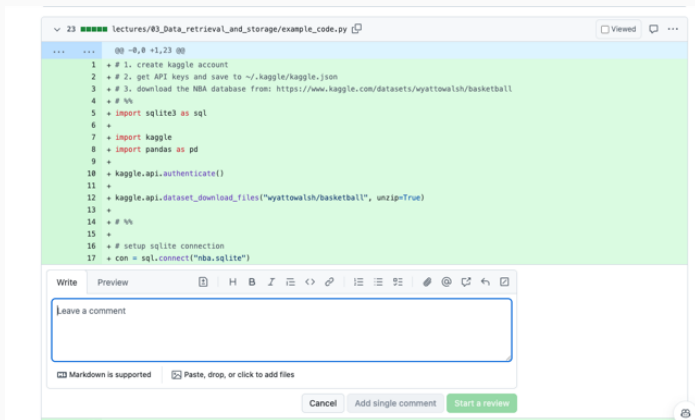
Merge pull request You can also open this in GitHub Desktop or view command line instructions.

Add a comment

Write Preview

Add your comment here...

PR reviews let you check the code of your co-workers



Use Git and GitHub for your problems set

For each problem set you will work in groups. When indicated, you work with repos that we provide on the GitHub page for this course. A team member has to:

- Go to <https://github.com/orgs/cambridge-FDS/repositories>
- Click on the repo for the problem set, e.g. "titanic"
- Fork the repo

Then, all team members:

- Clone the forked repo from that team members' GitHub
- Allocate tasks for each other
- Create a branch for your task
- Create a Pull Request (PR)
- Work on your task and commit and push frequently!
- Once you are done, ask for a PR review
- When everyone agrees, merge the branch into main

In the submission, please share the link to the repo you worked on.